

Ex.No.10 Use Ghidra to disassemble and analyze the malware code.

Project Objectives

- To provide a hands-on guide to disassembling and analyzing malware with Ghidra.
 - To enable readers to recognize common malware functionalities, such as persistence mechanisms, anti-analysis techniques, and network communications.
 - To equip users with the skills to interpret low-level assembly code and understand high-level behaviours.
-

GitHub Repository Components

A GitHub repository for this project could include:

1. README.md

- **Overview:** Brief description of Ghidra, its installation, and how it is used for malware analysis.
- **Requirements:** List of requirements, including Ghidra, a VM setup, and any sample binaries (preferably benign samples or detailed steps to safely acquire controlled malware samples).
- **Project Outline:** A breakdown of the tutorial steps, linking to various sections or files within the repository.

2. Tutorial Steps (Markdown or Jupyter Notebooks)

- **Step-by-step instructions, broken down into digestible sections, such as:**
 - **Environment Setup:** Instructions on setting up a virtualized and isolated environment.
 - **Initial Analysis:** How to load a binary into Ghidra, perform auto-analysis, and identify entry points.
 - **Function Analysis:** Methods for identifying and analyzing critical functions, decompiling code, and using Ghidra's cross-referencing features.
 - **String and Import Analysis:** How to locate and interpret strings and imports related to malicious behavior.
 - **Advanced Techniques:** Dynamic analysis, control flow graphs, custom scripting with Ghidra's Python or Java capabilities, and obfuscation detection.

3. Example Scripts (Python or Java)

- **Scripts to automate repetitive analysis tasks, such as:**
 - **Extracting and listing all string references.**

- Identifying network-related functions or common indicators of malware.
 - Labeling functions and data segments based on known patterns (e.g., file system manipulation, registry edits).
 - Script Examples: Python or Java code snippets demonstrating API usage in Ghidra (using Ghidra's Headless Analyzer, for example, for automation).
- 4. Sample Data (Benign Binary or Hex Dump)**
- Due to safety and ethical concerns, include only safe, benign binaries or a hex dump example rather than live malware. This can allow users to learn and practice without the risk of executing malicious code.
 - Sample binaries designed to exhibit typical malware-like behaviors, such as:
 - Executing system calls.
 - Writing to files.
 - Sending network data.
 - Alternatively, a dummy binary generator script that creates binaries with similar structures to malware but without harmful payloads.
- 5. Documentation and Comments**
- Include explanations of the code within each script, detailing what each function does.
 - Inline comments in disassembly views, providing readers with insights into assembly commands, control flow structures, and the decompiled code.
- 6. Report Template**
- A reporting template for documenting findings, which could be in Markdown or PDF format. The template should cover:
 - **Summary of Analysis:** Brief description of the suspected malware.
 - **Function Analysis:** Summary of the key functions, their purpose, and the findings.
 - **Behavioral Indicators:** Summary of malware behaviors, such as persistence, file system manipulation, or network communications.
 - **Recommendations:** Suggestions on further actions, such as containment, eradication, or further analysis.

Example Repository Structure

plaintext

ghidra-malware-analysis-project/

├── README.md

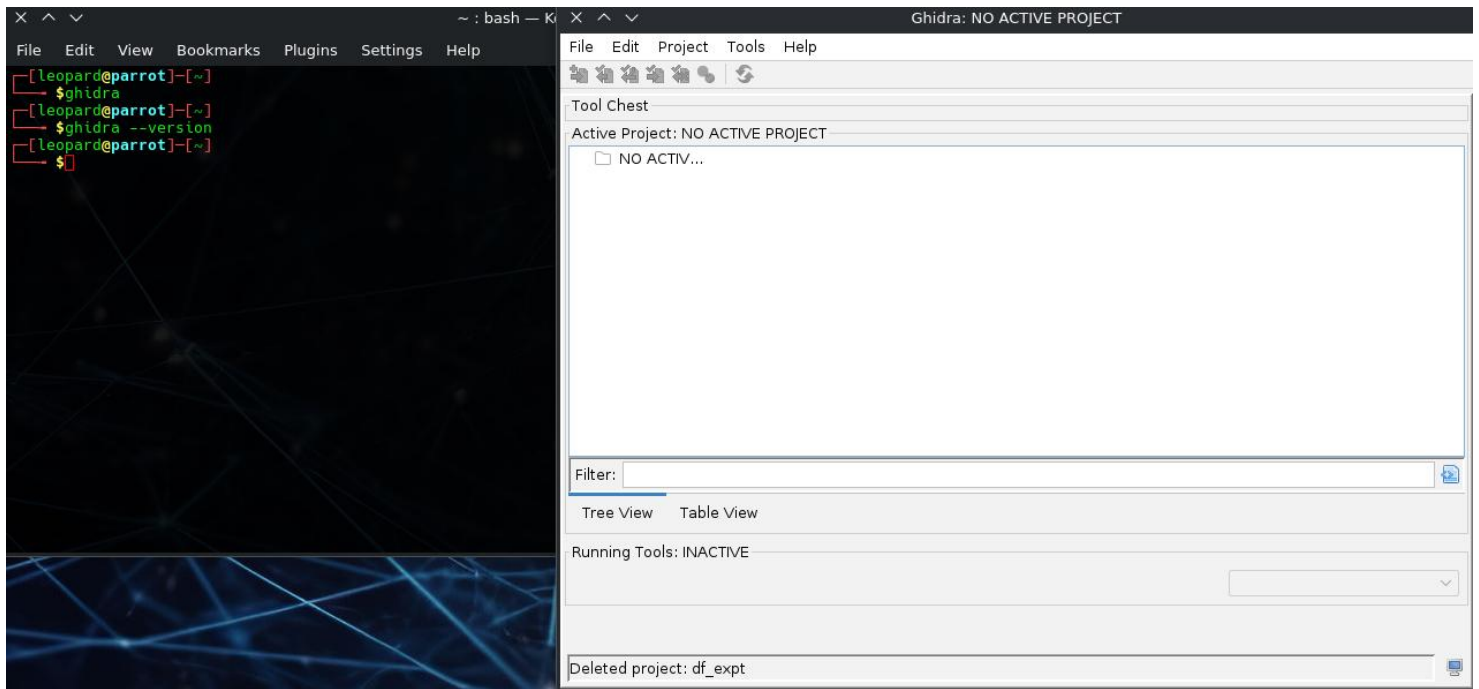
├── docs/

```
| | └─ Tutorial_Step1.md
| | └─ Tutorial_Step2.md
| | └─ Tutorial_StepN.md
| └─ scripts/
| | └─ extract_strings.py
| | └─ network_analysis.py
| | └─ label_functions.py
| └─ samples/
| | └─ benign_sample1.bin
| | └─ benign_sample2.hex
| └─ templates/
| | └─ analysis_report_template.md
└─ screenshots/
    └─ function_graph_example.png
    └─ decompiler_example.png
```

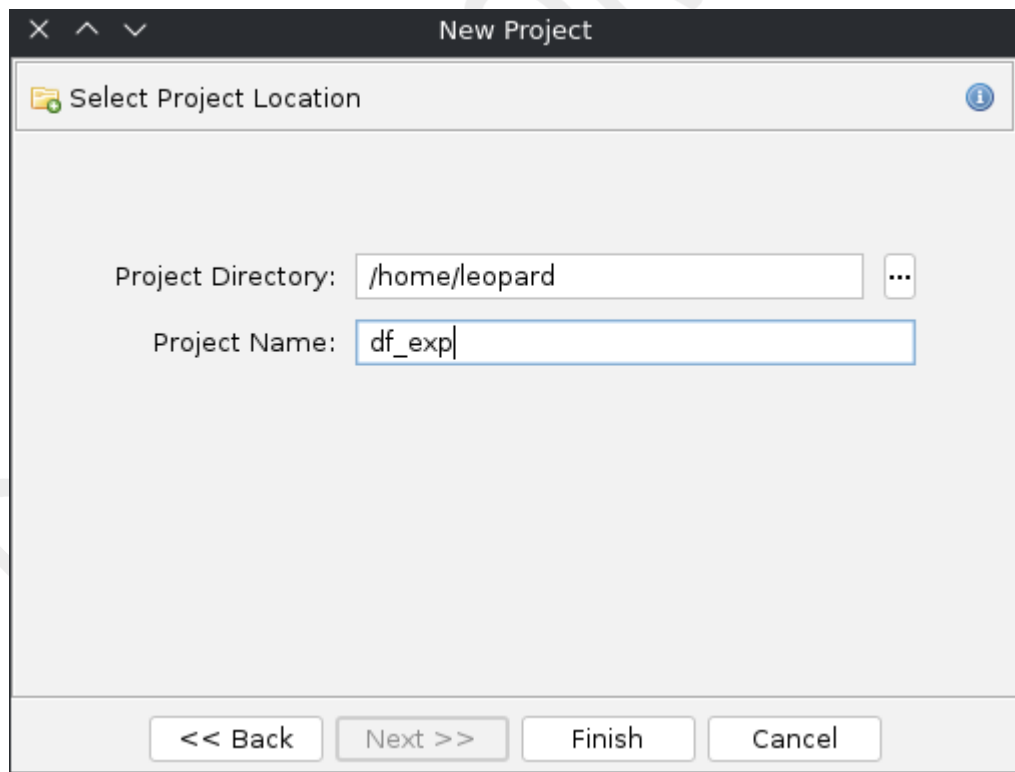
Project Development and Future Work

- **Expand Scripts:** Continue adding Ghidra scripts to automate complex analysis tasks.
- **Community Contributions:** Encourage community members to contribute additional benign samples, analysis scripts, and documentation improvements.
- **Advanced Topics:** Include guides for advanced techniques, such as unpacking obfuscated code, identifying encryption routines, and using the P-Code emulator for runtime analysis.

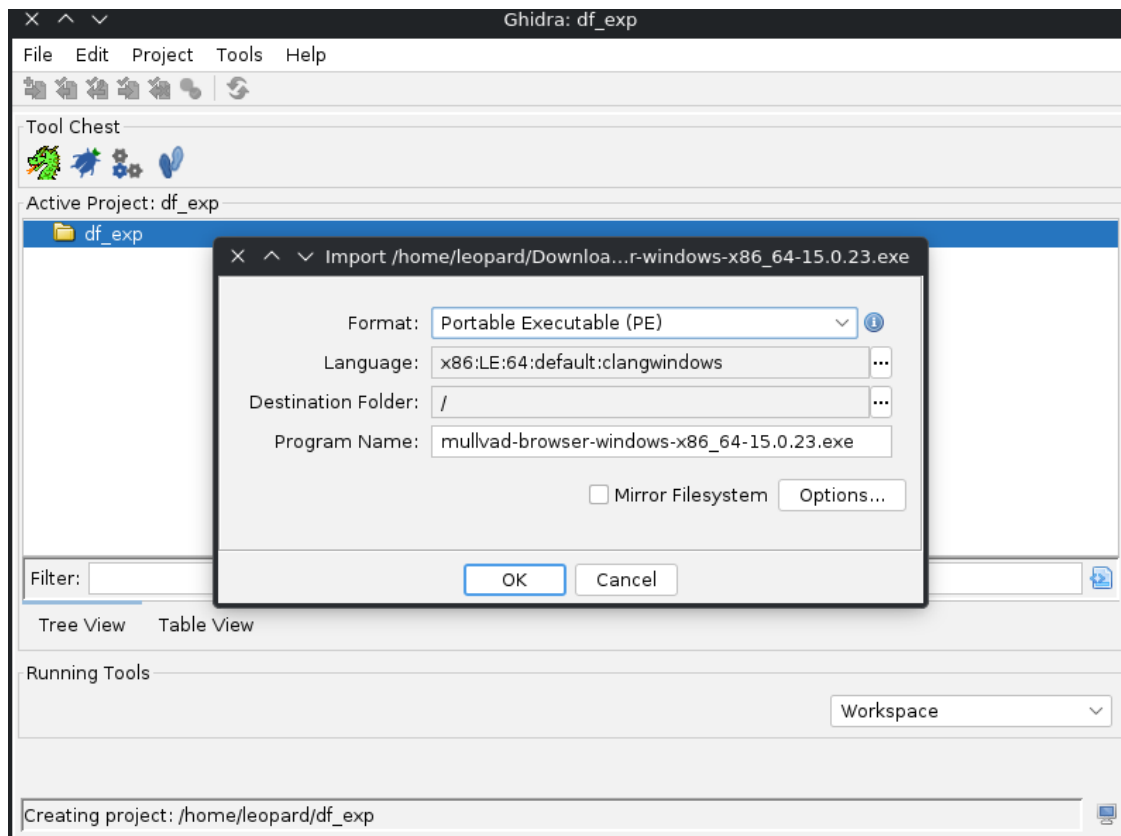
Step 1: Ghidra



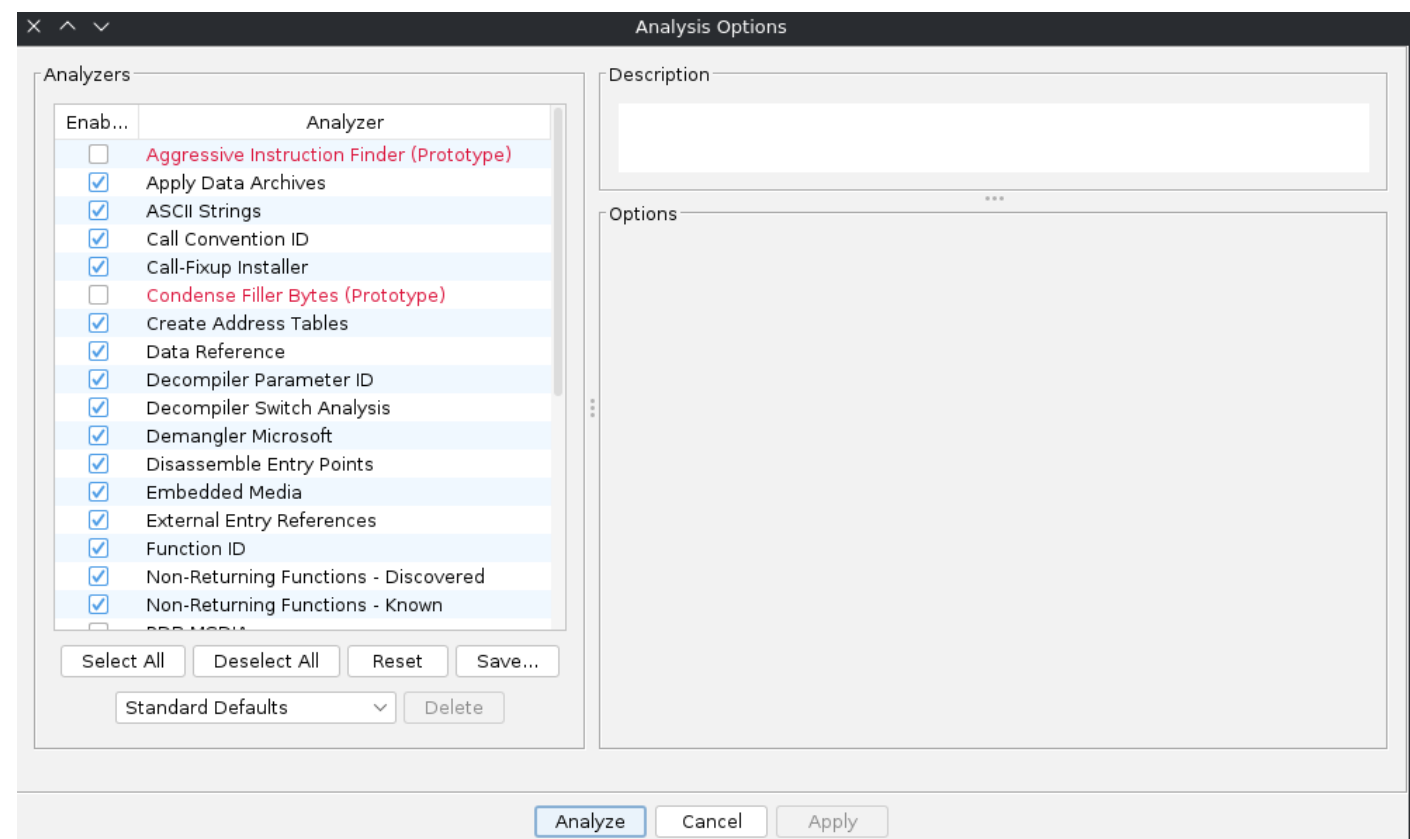
Step 2: Creating the project



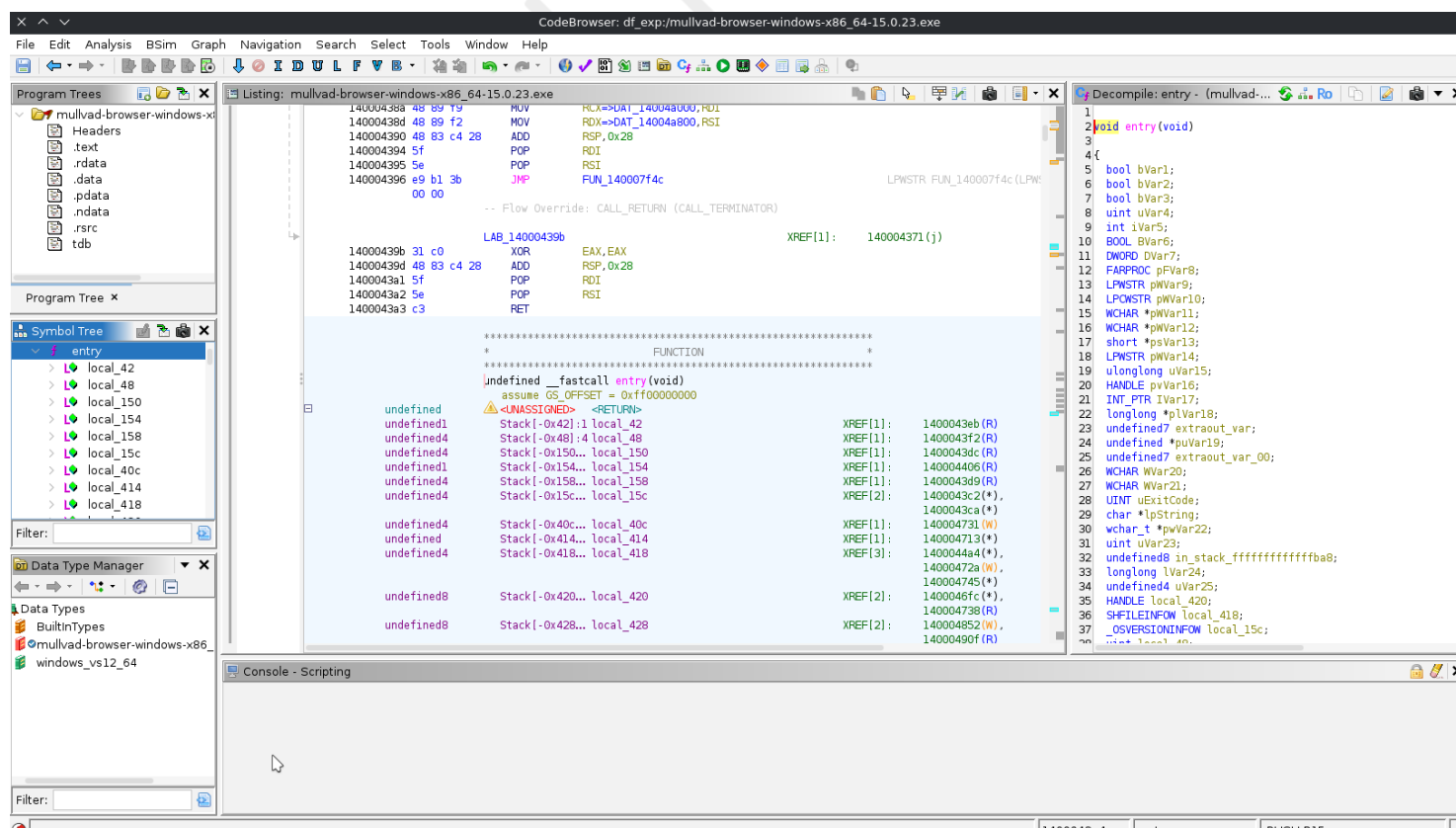
Step 3: Importing the .exe file



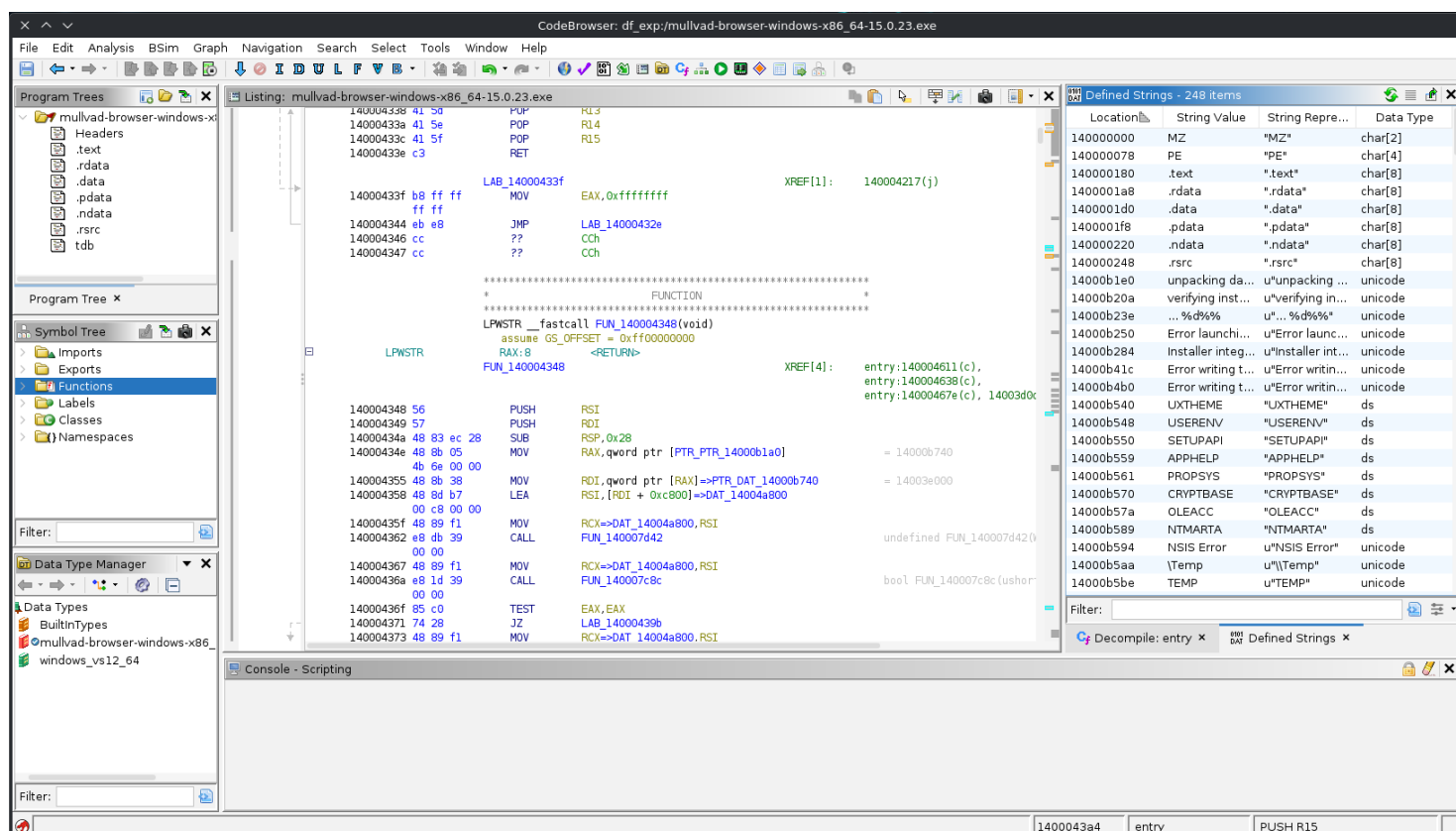
Step 4:Analyzing the file



Step 5:Decompiler



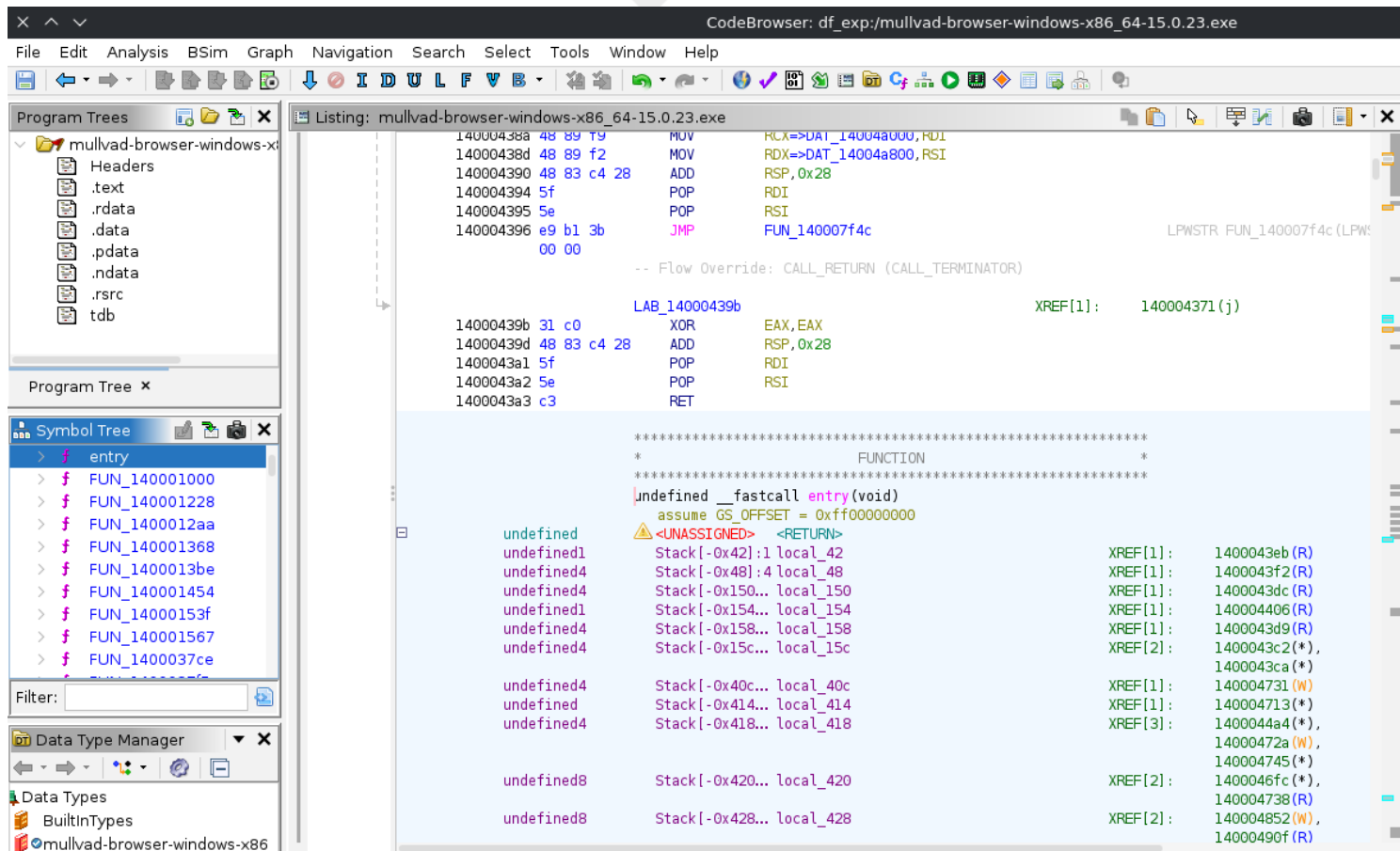
Step 6: Strings



The screenshot shows the CodeBrowser interface for the file `df_exp/multivad-browser-windows-x86_64-15.0.23.exe`. The main window displays assembly code for the function `LPWSTR __fastcall FUN_140004348(void)`. The code includes instructions like `POP R13`, `POP R14`, `POP R15`, `RET`, `MOV EAX, 0xffffffff`, `JMP LAB_14000432e`, and `CALL RAX, qword ptr [PTR_PTR_14000b1a0]`. The right-hand pane shows a list of defined strings, including "MZ", "PE", ".text", ".idata", ".data", ".pdata", ".ndata", ".rsrc", "unpacking da...", "verifying inst...", "Error launch...", "Installer integ...", "Error writing t...", "Error writin...", "UXTHEME", "USERENV", "SETUPAPI", "APPHelp", "PROPSYS", "CRYPTBASE", "OLEACC", "NTMARTA", "NSIS Error", "Temp", and "TEMP".

Location	String Value	String Repre...	Data Type
140000000	MZ	"MZ"	char[2]
140000078	PE	"PE"	char[4]
140000180	.text	".text"	char[8]
1400001a8	.idata	".idata"	char[8]
1400001d0	.data	".data"	char[8]
1400001f8	.pdata	".pdata"	char[8]
140000220	.ndata	".ndata"	char[8]
140000248	.rsrc	".rsrc"	char[8]
14000b1e0	unpacking da...	u"unpacking ..."	unicode
14000b20a	verifying inst...	u"verifying in..."	unicode
14000b23e	...%d%%%	u"...%d%%%"	unicode
14000b250	Error launch...	u"Error launc..."	unicode
14000b284	Installer integ...	u"Installer int..."	unicode
14000b41c	Error writing t...	u"Error writin..."	unicode
14000b4b0	Error writing t...	u"Error writin..."	unicode
14000b540	UXTHEME	"UXTHEME"	ds
14000b548	USERENV	"USERENV"	ds
14000b550	SETUPAPI	"SETUPAPI"	ds
14000b559	APPHelp	"APPHelp"	ds
14000b561	PROPSYS	"PROPSYS"	ds
14000b570	CRYPTBASE	"CRYPTBASE"	ds
14000b57a	OLEACC	"OLEACC"	ds
14000b589	NTMARTA	"NTMARTA"	ds
14000b594	NSIS Error	u"NSIS Error"	unicode
14000b5aa	Temp	u"Temp"	unicode
14000b5be	TEMP	u"TEMP"	unicode

Step 7: Functions



The screenshot shows the CodeBrowser interface for the file `df_exp/multivad-browser-windows-x86_64-15.0.23.exe`. The main window displays assembly code for the function `LPWSTR FUN_140007f4c(LPWSTR)`. The code includes instructions like `MOV RCX, >DAT_140048000, RDI`, `MOV RDX, >DAT_140048000, RSI`, `ADD RSP, 0x28`, `POP RDI`, `POP RSI`, and `JMP FUN_140007f4c`. The right-hand pane shows a list of defined functions, including `entry`, `FUN_140001000`, `FUN_140001228`, `FUN_1400012aa`, `FUN_140001368`, `FUN_1400013be`, `FUN_140001454`, `FUN_14000153f`, `FUN_140001567`, and `FUN_1400037ce`.

Location	String Value	String Repre...	Data Type
140000000	MZ	"MZ"	char[2]
140000078	PE	"PE"	char[4]
140000180	.text	".text"	char[8]
1400001a8	.idata	".idata"	char[8]
1400001d0	.data	".data"	char[8]
1400001f8	.pdata	".pdata"	char[8]
140000220	.ndata	".ndata"	char[8]
140000248	.rsrc	".rsrc"	char[8]
14000b1e0	unpacking da...	u"unpacking ..."	unicode
14000b20a	verifying inst...	u"verifying in..."	unicode
14000b23e	...%d%%%	u"...%d%%%"	unicode
14000b250	Error launch...	u"Error launc..."	unicode
14000b284	Installer integ...	u"Installer int..."	unicode
14000b41c	Error writing t...	u"Error writin..."	unicode
14000b4b0	Error writing t...	u"Error writin..."	unicode
14000b540	UXTHEME	"UXTHEME"	ds
14000b548	USERENV	"USERENV"	ds
14000b550	SETUPAPI	"SETUPAPI"	ds
14000b559	APPHelp	"APPHelp"	ds
14000b561	PROPSYS	"PROPSYS"	ds
14000b570	CRYPTBASE	"CRYPTBASE"	ds
14000b57a	OLEACC	"OLEACC"	ds
14000b589	NTMARTA	"NTMARTA"	ds
14000b594	NSIS Error	u"NSIS Error"	unicode
14000b5aa	Temp	u"Temp"	unicode
14000b5be	TEMP	u"TEMP"	unicode

Step 8:Exporting the results

